

Concludo — Product Requirements Document

v1: Revenue Recognition (ASC 606) & Capitalized Commissions (ASC 340-40) Judgment Engine

Product Requirements Document — Build Handoff

v1.0

Table of Contents

1. Product Summary

What this is. Concludo is an “accounting intelligence layer” for judgment-heavy technical accounting topics that do not live cleanly inside an ERP: revenue recognition (ASC 606) and capitalized sales commissions (ASC 340-40) for v1, with equity method accounting explicitly deferred to a later phase.

Who it's for. Controllers and CFOs at early-stage/mid-market companies (SaaS-style B2B, initially) who currently perform this analysis manually in spreadsheets, and who need audit-defensible documentation, not just a faster spreadsheet.

What it is not. This is not an ERP or system of record. It does not replace NetSuite/Intacct/etc. It sits alongside the ERP, ingests contracts and policy documents that the ERP doesn't model, produces accounting judgments and a technical memo, and exports journal entries back to the ERP for posting.

Core product thesis. The hard part of this problem is not arithmetic — it's judgment (is this performance obligation distinct? what's the standalone selling price? does this modification get separate-contract treatment?). The product's job is to propose defensible judgments with cited reasoning, route them to a human for approval, and produce audit-ready documentation as a byproduct — not to auto-post numbers a Controller can't explain to their auditor.

2. Design Principles (Non-Negotiable)

These principles should govern every implementation decision below. Do not compromise on these for expedience:

1. **Provenance on every extracted fact.** Every fact pulled from a contract must carry a pointer back to its exact source location (page/clause/quote). No fact should exist in the system without a way to answer “where did this come from?”
 2. **Judgments are immutable and versioned, never edited in place.** When a judgment changes (reassessment, override, new information), create a new version that supersedes the old one. The old version and its original reasoning must remain permanently retrievable. This is what makes the audit trail real rather than decorative.
 3. **Deterministic math never runs through the LLM.** SSP allocation arithmetic, amortization schedules, journal entry calculations — all plain, testable, deterministic code. The LLM’s job is to propose judgments (distinctness, method selection, modification type, variable consideration treatment); code executes the resulting math.
 4. **Confidence tiering is real and behavior-changing, not cosmetic.** A judgment made with no observable comparable data (`benchmark_only`) must produce visibly different, more cautious output — explicit disclosure language, lower prominence, flagged for reassessment — than one backed by ten comparable contracts (`observed_high`). The system must actively resist producing confident-sounding answers when it doesn’t have the evidence to support confidence.
 5. **Billing/cash timing is never revenue recognition timing.** Milestone invoicing, prepayment schedules, and commission payment timing are payment mechanics. They must never be conflated with when revenue or expense is actually recognized.
 6. **When the system isn’t sure, it escalates — it does not guess.** Contradictory contract terms, illegible text, missing required fields, and conflicting source documents must produce explicit flags routed to a human, not a silently-picked best guess.
 7. **Every judgment must show its rejected alternatives.** A judgment’s reasoning trace should state not just the conclusion but why the other plausible treatments were rejected. This is what makes a memo defensible to an auditor who will independently re-derive the analysis.
-

3. Scope

3.1 In Scope for v1

- Contract ingestion and structured extraction (PDF/text in, structured facts + provenance out)
- Policy management: upload existing policy OR generate a starter policy from the standard + curated benchmark data; both converge into one Policy object
- Gap detection: **compliance gaps** (policy vs. standard) and **coverage gaps** (policy vs. actual contract scenarios) only. Consistency gap detection (AI-proposal-vs-policy mismatch auto-flagging) is explicitly deferred.

- Full ASC 606 judgment chain: contract identification/combination, performance obligation identification (distinctness), transaction price determination (including variable consideration), standalone selling price (SSP) allocation, recognition pattern determination
- Contract modification handling: the full three-way decision tree (separate contract / termination+new contract / cumulative catch-up)
- ASC 340-40 commission judgment chain: incremental cost determination, practical expedient check, capitalization, and amortization period determination (including the commensurate-renewal-rate test that can extend amortization beyond the initial term)
- SSP cold-start handling: confidence tiering (observed_high / observed_low / benchmark_only) and a curated benchmark data layer for when no company-specific comparable data exists
- Billing schedule extraction (contract-derived expected invoicing timing and amounts) and reconciliation against actual invoice data imported from the customer's billing system or ERP
- Contract asset (unbilled revenue) and contract liability (deferred revenue) computation and roll-forward, netted per contract per ASC 606-10-45
- Technical accounting memo generation (composed from approved judgments), including sensitivity notes and confidence disclosures; memo staleness flagging when underlying judgments change
- Human review/approval workflow: the Judgment Review Card, contract-grouped queue, exception queue, override with mandatory reason capture
- Deferred revenue schedule, commission amortization schedule, and JE batch generation, exportable as CSV or to one target ERP integration

3.2 Explicitly Out of Scope for v1 (Deferred)

- Cross-customer data aggregation / benchmark pooling ("Benchmark Pro" tier) — requires customer volume for statistically valid minimum-n thresholds and requires privacy/legal review before build. Benchmark layer in v1 uses public/licensed data only.
- Multi-ERP integration framework — build one integration well (target: NetSuite or Intacct) before generalizing.
- Configurable multi-threshold approval routing (dollar-based routing rules) — schema should reserve a reviewer chain concept, but the UI/config layer is not built in v1.
- Consistency gap detection (automated AI-vs-policy mismatch flagging) — refinement layer on top of a working judgment engine.
- Equity method accounting (ASC 323) and any other accounting domain beyond rev rec / commissions.

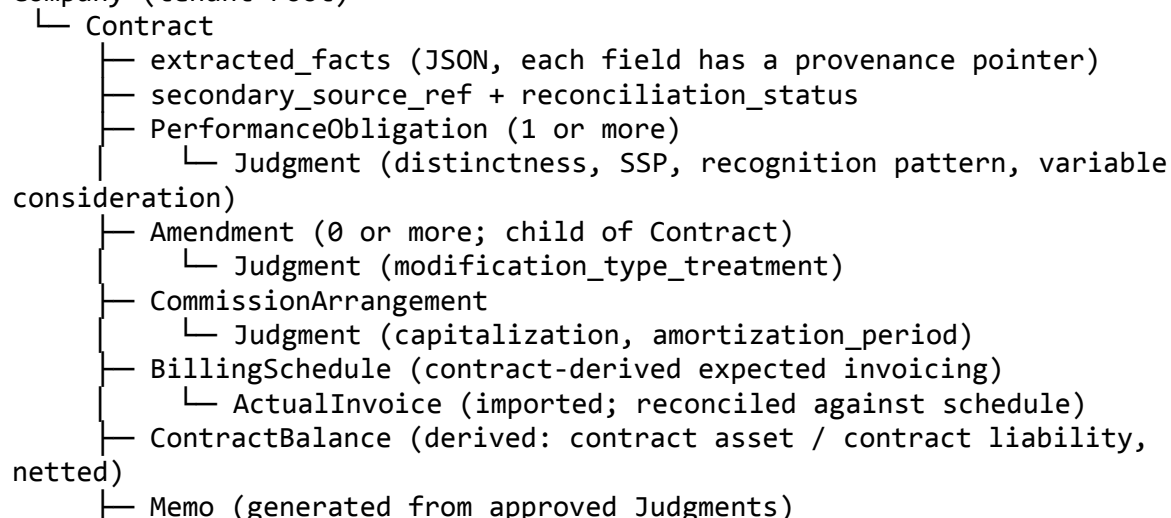
- A standalone SSP rate-card-building tool — customers without a rate card flow through the cold-start/benchmark path instead.
- Live CPQ/CRM API integrations — v1 supports manual upload of secondary documents with mismatch flagging, not live sync.
- **Invoice generation, accounts receivable, collections, dunning, and payment processing.** Concludo consumes billing data as an input to revenue recognition; it does not originate invoices or handle cash. The customer's existing billing system remains the system of record for invoicing. This boundary is deliberate: it preserves the "sits alongside your existing stack" positioning and avoids competing with contract-to-cash platforms.

The specific blockers on invoice generation are worth recording, since the boundary will be revisited: (a) **sales tax/VAT determination** is a separate discipline requiring a dedicated tax engine integration (Avalara, Anrok, Stripe Tax) — an incorrectly taxed invoice creates direct liability for the customer and is a materially worse failure mode than a debatable accounting judgment; (b) **invoice delivery** to enterprise buyers typically requires submission through AP portals (Coupa, Ariba, Tungsten), each its own integration; (c) invoices must still land in the ERP's **AR subledger**, where aging and cash application live — so generating them elsewhere trades a matching problem for a sync problem; (d) invoice numbering sequences, credit memos, voids, rebills, mid-cycle proration, and multi-currency each carry their own correctness requirements. None of these are insurmountable, but together they constitute a different product.

4. Core Data Model

4.1 Entity Overview

Company (tenant root)



└─ Schedule / JournalEntry (derived from approved Judgments only)

Policy (versioned, per Company, per standard)

└─ gap_flags (compliance / coverage)

BenchmarkDataset (versioned, curated, public/licensed data in v1)

4.2 Entity Specifications

Company

Tenant root. id, name, industry_segment.

Contract

- id, company_id, source_file, effective_date, term_end_date, status (draft / active / amended / terminated)
- extracted_facts: structured JSON. **Every individual fact must carry:** {value, source: {page, clause_text_span, quote}, confidence: high|medium|low}
- secondary_source_ref: link to any secondary document (e.g., CRM/CPQ export); reconciliation_status: matched | conflicting | not_linked
- related_contracts: self-referential array, used for contract-combination judgments and for linking “separate contract” modifications back to a parent relationship for reporting only (never for accounting mechanics)
- amendments: ordered array of child Amendment records

PerformanceObligation

- id, contract_id, description
- is_distinct: boolean — **this is a judgment output, never a raw/assumed field**
- recognition_pattern: { type: point_in_time | over_time, method }
- ssp: { method: market_assessment | cost_plus_margin | residual, value, support_refs, confidence_tier }
- allocated_price: derived from SSP allocation across all POs in the contract
- start_date, end_date — note: a PO can have a start date later than the contract’s effective date (see Contract 4 in the test suite) without this being a modification

Policy

- id, company_id, standard (e.g., ASC_606, ASC_340_40)
- version, source: uploaded | system_generated | hybrid, effective_date
- provisions: array of { determination_type, company_position, standard_citation }
- gap_flags: array of { type: compliance | coverage, description, related_contract_id (nullable), status }

Judgment — the central entity

- id, contract_id, performance_obligation_id (nullable — some judgments are contract-level, e.g. contract combination)

- `judgment_type`: enum — `contract_combination`, `distinctness`, `ssp_method`, `recognition_pattern`, `variable_consideration`, `modification_type`, `commission_capitalization`, `amortization_period`
- `standard_ref`: specific ASC citation(s)
- `policy_ref`: nullable FK to the Policy provision governing this determination
- `contract_evidence`: array of provenance pointers supporting the conclusion
- `ai_proposed_conclusion`: { `value`, `reasoning_steps`: [...], `rejected_alternatives`: [{`alternative`, `why_rejected`}] }
- `confidence_tier`: `observed_high` | `observed_low` | `benchmark_only`
- `benchmark_source_id`: nullable FK to `BenchmarkDataset`
- `human_decision`: { `approved_by`, `decision`: `accepted`|`overridden`, `override_reason` (required if `overridden`), `timestamp` }
- `sensitivity_note`: what changes if this judgment had gone the other way
- `status`: `proposed` | `under_review` | `approved` | `superseded`
- **Never updated in place.** A reassessment creates a new Judgment row with `status`: `superseded` on the old row and a link to its replacement.

Amendment

- `id`, `contract_id`, `effective_date`, `source_document` (own extraction + provenance, same pipeline as a Contract)
- `modification_type_judgment_id`: FK to a Judgment of type `modification_type`
- `superseded_performance_obligations`: populated only for the “termination + new contract” outcome
- `catch_up_adjustment`: { `judgment_id`, `amount`, `calculation_basis`, `recorded_period` } — populated only for the “cumulative catch-up” outcome

CommissionArrangement

- `id`, `contract_id`, `payee`, `total_commission`, `payment_schedule`
- `capitalization_judgment_id`: FK to Judgment (incremental cost determination)
- `amortization_period_judgment_id`: FK to Judgment (commensurate-renewal-rate test and resulting period)
- `amortization_schedule`: derived, mechanical, computed only once both judgments are approved

BillingSchedule

- `id`, `contract_id`, `amendment_id` (nullable — amendments can alter billing terms)
- `external_system_ref`: nullable — reserved for the v2 billing-schedule-push direction (Section 9). Present but unused in v1; ensures the schedule can later become an authored, pushable object rather than a read-only extraction artifact, without a migration.
- `billing_basis`: `advance` | `arrears` | `milestone` | `usage_actual` | `hybrid`
- `scheduled_invoices`: array of { `due_date`, `amount`, `description`, `basis`, `provenance` } — **derived from contract extraction**, since billing terms are stated

explicitly in substantially all contracts. Each scheduled invoice carries a provenance pointer like any other extracted fact.

- Note: a BillingSchedule does not necessarily map one-to-one to PerformanceObligations. A single invoice may cover multiple POs, and a single PO may be invoiced across many periods. Model the relationship as many-to-many, not nested.
- For usage-based components, the schedule is **expected/estimated** until actual usage is known; flag these entries as estimated rather than fixed.

ActualInvoice

- id, contract_id, invoice_date, invoice_number, amount, line_items
- source: manual_upload | erp_import | billing_system_import
- reconciliation_status against BillingSchedule: matched | variance | unmatched
- Variances route to the exception queue (same pattern as secondary-source contract mismatches), not silently overwritten. The contract-derived schedule represents what *should* have been billed; ActualInvoice represents what *was* billed. Both are retained; neither is authoritative over the other without human resolution.

ContractBalance (derived)

- Computed per contract, per period: revenue_recognized_to_date, amount_invoiced_to_date
- contract_asset (unbilled revenue): recognized revenue in excess of amounts invoiced
- contract_liability (deferred revenue): amounts invoiced in excess of recognized revenue
- **Netting requirement:** per ASC 606-10-45-1 through 45-5, contract assets and contract liabilities are presented **net at the contract level** — a single contract produces either a net contract asset or a net contract liability in a given period, never both simultaneously. The computation must net at contract level before any presentation or roll-up, not sum gross balances across POs.
- Unbilled receivables vs. contract assets: where an unconditional right to consideration exists (only the passage of time is required), the balance is a **receivable**, not a contract asset. The system should distinguish these, as the presentation differs.
- Derived strictly from approved Judgments plus ActualInvoice/BillingSchedule data — never manually entered.

Memo

- id, contract_id, generated_from_judgments: ordered list of Judgment IDs
- narrative_sections: generated text, structured to mirror the ASC 606 five-step model + ASC 340-40 analysis
- version, status: draft | finalized | stale

- **Must flip to stale, not silently stay correct, if any underlying Judgment is superseded after the memo was generated.**

Schedule / JournalEntry

- deferred_revenue_schedule, commission_amortization_schedule — derived only from approved Judgments
- je_batch: export object, with its own approval step before posting to the ERP

BenchmarkDataset (v1: public/licensed data only)

- id, source_description, effective_date_range, applicable_segment, data (versioned)
-

5. Functional Requirements by Module

5.1 Ingestion & Extraction

Requirement: Given a contract document (PDF or text), extract a structured fact set matching the `Contract.extracted_facts` schema, with mandatory provenance and confidence per field.

Pipeline stages: 1. Document normalization (OCR/layout-aware parsing if scanned; preserve page/position data) 2. LLM-based structured extraction, schema-constrained (JSON/tool-use output, not free text) — every field returned with source span and self-reported confidence 3. Deterministic validation pass (non-LLM): do line items sum to stated totals? Are dates internally consistent? Are required fields present? Route failures to a human review queue before anything reaches the Judgment engine. 4. Field-level confidence scoring, informed by both the model's self-report and structural signal (e.g., extracted from a formatted table vs. free-flowing prose correlates with reliability) — fields below threshold route to human confirmation. 5. Human confirmation step, distinct from Judgment review: side-by-side of extracted facts against the highlighted source document, for any flagged field.

Secondary source reconciliation: if a secondary document (CRM/CPQ export) is provided, compare against extracted contract facts. Flag mismatches; do not silently prefer one source. If the contract has an explicit precedence clause (e.g., "Exhibit A controls as to fees and quantities"), the system must locate and apply that clause, not assume its own default precedence.

Multi-document contracts: when a contract references or is bundled with amendments/exhibits/order forms with different execution dates, extract each with its own provenance and reconcile explicitly.

Escalation over guessing: if the contract contains internally contradictory terms (e.g., two different stated term lengths with no resolving amendment), the system must **not** silently

pick one. It produces a blocking extraction conflict, routed to human resolution, and does not proceed automatically into the Judgment engine.

Acceptance criteria: validated against the 15-contract test suite (Section 8). In particular: Contract 12 (MSA/Order Form numeric mismatch) must correctly apply the contract's stated precedence clause and correctly identify the CRM record as stale. Contract 15 (adversarial) must produce a blocking exception rather than a confident guess on term length, and must not fabricate values for the illegible/missing fields.

5.2 Policy Management & Gap Detection

Requirement: Support two entry points into the same Policy object: - **Upload path:** parse an existing policy document into structured provisions, each mapped to a `determination_type` and the standard citation it's grounded in. - **Generate path:** in the absence of an uploaded policy, generate a starter policy from the standard plus the BenchmarkDataset, explicitly labeled as system-generated and provisional, recommended as the customer's policy of record pending their review/adoption.

Gap detection (v1 scope: compliance and coverage only): - **Compliance gap:** a provision in the customer's policy conflicts with the standard itself. - **Coverage gap:** a contract scenario arises (e.g., a new deliverable type) that the existing policy does not address at all. - Gap flags are first-class, persistent records (not just a one-time surfaced warning), with a status that can be tracked and resolved over time.

Explicitly deferred: consistency gap detection (automated flagging when an AI-proposed judgment differs from what the policy would predict).

5.3 ASC 606 Judgment Engine

Implement the following judgment types, each producing a Judgment record with the standard structure (proposed conclusion, reasoning steps, rejected alternatives, evidence, confidence tier):

a) Contract identification/combination — determine whether related documents constitute one contract or several; relevant when a customer has multiple concurrent or closely-timed agreements.

b) Distinctness (performance obligation identification) — apply the two-part test: (1) capable of being distinct (customer can benefit from the good/service on its own or with readily available resources) and (2) separately identifiable (not significantly integrated with, modified by, or dependent on other promises in the contract). Both prongs must be satisfied for a "distinct" conclusion. Reasoning must cite the specific contract language supporting or refuting each prong.

c) Transaction price / variable consideration — identify all variable consideration (usage-based fees, rebates, refundable termination rights tied to contingent future events, volume discounts). For each: determine whether it is genuinely variable consideration (contingent on a future uncertain event) as opposed to a standard performance-assurance term (e.g., a refund tied to the vendor's own breach is not variable consideration in the

same sense as a refund tied to an independent future contingency). Select an estimation method (expected value vs. most likely amount) with reasoning for the choice, and apply the constraint (recognize only to the extent it's probable a significant reversal will not occur).

d) Standalone selling price (SSP) allocation — see Section 5.4 for full detail, as this is the highest-complexity judgment in the engine.

e) Recognition pattern — determine point-in-time vs. over-time recognition per the relevant indicators (control transfer, simultaneous receipt and consumption of benefit, asset with no alternative use plus a right to payment for performance completed to date). Must correctly decouple recognition timing from billing/milestone timing.

5.4 SSP Allocation Logic (Detailed)

Method selection, in this priority order of applicability, not a fixed hierarchy: 1. **Adjusted market assessment** — usable only if genuine third-party/market comparable pricing exists (rare in practice for this buyer segment). 2. **Expected cost plus margin** — primary fallback for services-type POs when cost data is available. 3. **Residual approach** — permitted **only** when the standalone price for a specific PO is highly variable or uncertain (ASC 606-10-32-34(c)), not merely because it's the easiest math. The system must explicitly test and state why residual is or is not permitted before applying it.

Observable price corpus: maintain an ObservablePriceHistory object per company_id + deliverable type, built by extracting pricing data (bundled or standalone) across all of a customer's ingested contracts. This corpus is the evidence base every SSP judgment should cite — it is more valuable than any single judgment and should be recomputed as new contracts are ingested.

Confidence tiering (mandatory, not optional): - observed_high: sufficient comparable data (target: 10+ comparable observations) - observed_low: some comparable data but below the high-confidence threshold (target: 1-9 observations) - benchmark_only: zero company-specific comparables; relies on the curated BenchmarkDataset

Cold-start handling: for benchmark_only judgments, the AI's role shifts from "estimate" to "recommend a defensible provisional starting position." Every memo section built from a benchmark_only judgment must include mandatory disclosure language stating the determination relied on benchmark data in the absence of company-specific pricing history, and that management should reassess as the company's own pricing history develops.

Reassessment/graduation: implement a ReassessmentTrigger concept — as a customer's own ObservablePriceHistory grows, previously benchmark_only or observed_low judgments that now have sufficient comparable data should be flagged for mandatory reassessment, not left stale indefinitely.

Residual-method failure handling: if applying the residual approach produces a negative or unreasonably low allocation, the system must **not** silently produce that result. It must

flag the residual-method failure explicitly and propose an alternative method (cost-plus or proportional adjustment), clearly marked as requiring human review.

5.5 Contract Modification Engine

Implement the three-outcome decision tree per ASC 606-10-25-10 through 13, as a `modification_type` Judgment on each Amendment:

Decision logic: 1. Are the additional goods/services **distinct**? If no → **cumulative catch-up** (Outcome C). 2. If distinct, does the additional price **reflect standalone selling price** (adjusted for circumstances)? If yes → **separate contract** (Outcome A). If no → **termination of old contract + creation of new contract** (Outcome B).

Outcome A — Separate contract: No impact on the original contract or its existing POs/schedules. The new elements are modeled as an independent Contract-like judgment chain, linked to the original only for relationship/reporting rollup.

Outcome B — Termination + new contract: Calculate the remaining (unrecognized) transaction price from the original contract as of the modification date (a mechanical calculation from the existing recognition schedule, not a new judgment). Combine with the new consideration. Reallocate the combined amount across all remaining POs based on relative SSP as of the modification date. Applied **prospectively only** — no restatement of amounts already recognized.

Outcome C — Cumulative catch-up: Recalculate the measure of progress and total transaction price for the affected PO as of the modification date. Recognize the difference between revenue that would have been recognized to date under the new terms and revenue actually recognized to date as an **immediate, isolated catch-up adjustment** in the period of modification — this figure must be clearly separated in the memo and JE output from routine periodic recognition, not buried inside it.

Every `modification_type` judgment's reasoning must explicitly state why the other two outcomes were rejected.

5.6 ASC 340-40 Commission Engine

Capitalization determination: is the commission an incremental cost of obtaining the contract (would not have been incurred but for successful execution)? Standard case: yes.

Practical expedient check: if the amortization period would be one year or less, the practical expedient permits immediate expensing. Must be explicitly checked and stated, not skipped.

Amortization period determination — the core judgment: - Default/standard case: amortize over the initial contract term, **if and only if** the renewal commission rate is commensurate with the initial commission rate (i.e., the initial commission is not, in substance, also compensating for anticipated renewals). - **Commensurate test:** compare initial commission rate to renewal commission rate. If the renewal rate is substantially lower and not reflective of the effort/value of securing a renewal, the test fails, and the

amortization period must extend to include anticipated renewal periods. - When extending the period: use historical renewal-rate data for comparable customer cohorts, subject to a policy-defined maximum (e.g., capped at initial term plus a defined number of renewal terms absent specific longer-period evidence), to maintain a defensible, conservative estimate. - **Cash payment timing (e.g., 50% at signing / 50% at a later date) never affects the amortization schedule.** The full commission asset is capitalized when the commission becomes payable/probable, and amortized from the contract's revenue-recognition start date. - Every amortization-period judgment involving a renewal-period extension must include an explicit sensitivity note showing how the monthly expense would change under a different reasonable renewal-period assumption.

5.7 Memo Generation

Generate a technical accounting memo per contract, composed directly from approved Judgment records (the Judgment Review Card and the memo section should be two renderings of the same underlying object, not separately maintained content).

Required structure: background/facts, ASC 606 five-step analysis (each step's conclusion, citation, and reasoning), ASC 340-40 commission analysis where applicable, an explicit list of judgments made with sensitivity notes, and confidence/disclosure language for any `benchmark_only` or `observed_low` determinations.

Staleness handling: if any Judgment feeding a finalized memo is later superseded, the memo's status must flip to stale automatically. It must not silently continue displaying as current/correct.

5.8 Human Review & Approval Workflow

Judgment Review Card — the atomic UI unit, rendering every Judgment type in a consistent shape: the question in plain language, the proposed conclusion, the reasoning chain (each step linked to its evidence), the evidence itself (clickable, linking to the highlighted source location), the confidence tier as a visible badge, the sensitivity note (collapsed by default), and three actions: **Accept**, **Override** (mandatory reason, permanently recorded), **Request more evidence** (routes for further research without forcing an immediate decision).

Queueing: - Default view: grouped by contract. - Exception queue: extraction mismatches, policy gap flags, stale `benchmark_only` judgments due for reassessment, memos flagged stale — kept separate from routine Judgment review. - Risk/confidence triage view: surface everything below `observed_high` first.

Override handling: mandatory reason capture, tied permanently to the superseded Judgment version. Prompt (not force) the user on whether the override should become a standing Policy provision, closing the loop between real decisions and policy evolution. Surface override patterns in aggregate (e.g., frequency and direction of overrides on a given judgment type) as a signal to the Controller/CFO.

Period lock: once a period is closed, its Judgments and Memos become read-only; any change must go through the supersede mechanism, never an in-place edit. Build this from v1, not retrofitted later.

Roles: v1 may use a single approver role in the UI, but the `human_decision` schema must support a reviewer chain structurally, even if unused in v1's UI/config layer.

5.9 Billing Schedule & Contract Balance Tracking

Requirement: Capture expected billing timing from the contract, reconcile against actual invoicing, and compute contract asset/liability balances as a derived output.

Billing schedule extraction. Billing terms are stated explicitly in substantially all contracts (e.g., “billed annually in advance, \$120,000 due at signing; \$120,000 due April 1, 2026”). Extract these into a `BillingSchedule` with the same provenance and confidence requirements as any other extracted fact. Handle the common variants: advance billing, arrears billing, milestone billing, prorated partial periods, and deferred-start components.

Critical principle (restated from Section 2): the billing schedule and the revenue recognition schedule are independent. Milestone invoicing does not drive recognition; a deferred-start performance obligation does not delay recognition of other obligations; usage billed in arrears produces revenue before invoicing. The system must never derive one schedule from the other.

Actual invoice reconciliation. Support import of actual invoice data (manual CSV upload in v1; ERP or billing system import where available). Compare against the contract-derived schedule and flag variances to the exception queue. Common legitimate causes of variance — mid-period amendments, credits, timing shifts, usage true-ups — should be surfaced for human resolution rather than auto-reconciled.

Actuals are a verification layer, not a prerequisite. This is a load-bearing design principle. Because the billing schedule is derived from the contract itself, contract balances are computable from expected billing alone. For customers who bill according to their contract terms — which is the normal case — expected equals actual. Importing actuals catches drift between what should have been billed and what was; it does not gate the balance computation. **The system must produce correct contract asset/liability balances for a customer who has imported zero invoice data**, marking those balances as projected rather than actual-verified. This means invoice import and matching can ship after v1 without blocking the core product.

Invoice-to-contract matching strategy. Concludo exists precisely because contract data does not live in the ERP, so invoices generally carry no contract identifier to join against. Matching must therefore be a tiered strategy, degrading gracefully:

1. **Single active contract per customer** — trivially resolved, and the majority case for the early-stage target buyer. Implement this first; do not over-engineer past it.
2. **Explicit reference match** — join on an order form number, subscription ID, sales order, or PO number appearing in both records. Every contract in the test suite

carries an Order Form number; this is the ideal key where it has been carried onto the invoice.

3. **Probabilistic match** on customer + amount + date proximity against the expected BillingSchedule. Propose the match with a confidence score and route to human confirmation using the same review pattern as Judgments. Never auto-accept a probabilistic match silently.
4. **Manual mapping, persisted as a rule.** Present unmatched invoices alongside candidate contracts for user assignment, then **persist the mapping** so it is not repeated each period. Most long-tail value lives here.

Matching edge cases (defer implementation, but do not let the schema preclude them): consolidated invoices covering multiple contracts require **line-level** rather than invoice-level matching; credit memos and rebills reduce invoiced amounts and must flow through to balances; the ERP customer record often differs from the contracting legal entity (parent vs. subsidiary); and following a modification treated as a separate contract, one customer will have two contracts with near-identical billing patterns.

Note on trigger: a contract liability arises when consideration is received **or becomes unconditionally due, whichever is earlier** — invoicing, not payment, is the trigger. Payment data is therefore not required for balance computation; it matters only for distinguishing a receivable from a contract asset.

Contract balance computation. For each contract, for each period: compute revenue recognized to date from approved Judgments and their resulting recognition schedules; compute amounts invoiced to date from ActualInvoice records (falling back to BillingSchedule where actuals are not yet available, clearly flagged as projected); derive the net contract asset or contract liability, netted at contract level per ASC 606-10-45. Distinguish unconditional receivables from contract assets.

Roll-forward presentation. Produce a period-over-period roll-forward of contract balances (opening balance, revenue recognized, amounts invoiced, closing balance) — this is a standard audit request and should be a first-class output, not something a Controller has to reconstruct.

5.10 Schedule & JE Export

Generate all schedules purely mechanically from approved Judgments plus billing data: - Revenue recognition schedule (per PO, rolled to contract and entity level) - Deferred revenue / contract liability schedule - Unbilled revenue / contract asset schedule - Commission capitalization and amortization schedule

Journal entry batch generation must cover the full set of entries these schedules imply — revenue recognition, movement in contract assets and contract liabilities, commission asset capitalization, and commission amortization expense. Cumulative catch-up adjustments arising from contract modifications must appear as clearly labeled, separately identifiable lines rather than being netted into routine periodic entries.

The `je_batch` requires its own approval step before posting, distinct from Judgment approval. v1 supports manual export (CSV) or a single target ERP integration (recommend NetSuite or Intacct — confirm based on first design-partner customer’s actual stack before building). ERP export is the terminal step of the product lifecycle: contract in, approved judgments, schedules and balances, journal entries out, posted in the customer’s ERP.

6. Non-Functional Requirements

- **Auditability by default:** every Judgment, every override, every superseded version must be permanently retrievable — this is a core requirement, not a “nice to have” logging feature.
 - **No silent data loss on correction:** the system must never overwrite a Judgment, extracted fact, or Memo version in place.
 - **Tenant isolation:** all data scoped strictly by `company_id`; no cross-tenant data visibility in v1 (aggregation is explicitly deferred and, when built, must compute-then-aggregate within tenant boundaries, never centralize raw data).
 - **Explainability as a rendering requirement, not just a data requirement:** the UI must always be able to answer “why did the system conclude this?” and “what would have to be different for the conclusion to change?” directly from stored data, without needing to re-run any model call.
-

7. Build Sequence (Recommended)

Given a solo builder using Claude Code, sequence work to front-load the riskiest unknowns rather than build in idealized dependency order:

1. **Extraction spike (throwaway script, not product code).** Build a minimal script: ingest each of the 15 test contracts (Section 8), extract structured facts via a schema-constrained prompt with mandatory provenance and confidence, and diff against the gold-standard answer keys. Do not proceed to schema/product work until this produces a clear picture of where extraction breaks. Contracts 12 and 15 are the pass/fail gate for “does the system escalate appropriately instead of guessing.”
2. **Minimal schema + one full judgment chain, script-only (no UI).** Implement Contract, PerformanceObligation, Judgment only. Run one contract through `distinctness` → `SSP` → `recognition pattern` → a markdown-rendered memo.
3. **Extend to commissions and modifications,** still script-only, using the same pipeline pattern. Add billing schedule extraction here as well — it is part of the same extraction pass and requires no new judgment logic, but it is the prerequisite for contract balance computation.

4. **Minimal review UI.** Build the Judgment Review Card and contract-grouped queue as a real (if rough) web app — accept/override/request-more-evidence, immutable versioning, exception queue.
5. **Policy object, gap detection, contract balance computation, schedules/JE export.**
6. **First real design-partner customer,** using their actual contracts.

Build the provenance and immutable-versioning requirements into the very first schema migration and extraction function — these are inexpensive now and expensive to retrofit once real data exists in the system.

8. Acceptance Test Suite

v1 must correctly handle the following 15 engineered test contracts (full contract text and gold-standard answer keys provided as companion documents: `Concludo_RevRec_Test_Suite.docx` and the accompanying source markdown files). Each contract isolates specific judgment(s):

#	Contract	Primary Judgment(s) Under Test
1	Meridian Health Systems	Baseline/control: single PO, no allocation needed
2	Brightwell Logistics	Distinctness (positive case); SSP with observable comparables
3	Corvus Manufacturing	Distinctness (negative case) — significant customization/integration
4	Halden Financial	Multiple POs at inception, one with a future start date (not a modification)
5	Rivergate Partners	SSP with zero comparables — benchmark-only tier
6	Solstice Retail Group	SSP allocation via residual method; permissibility check;

#	Contract	Primary Judgment(s) Under Test
		negative-residual failure handling
7	Nordholm Industries	Variable consideration estimation (expected value; rebate constraint)
8	Castellan Public Schools	Variable consideration from a refundable, contingency-based termination right
9	Brightwell Logistics — Mod A	Modification: separate contract treatment
10	Corvus Manufacturing — Mod B	Modification: termination of old + creation of new contract
11	Halden Financial — Mod C	Modification: cumulative catch-up adjustment
12	Piermont Analytics	Extraction/reconciliati on: multi-document conflict, precedence clause, stale CRM data
13	Meridian Health Systems (commission)	ASC 340-40 baseline: capitalize & amortize over contract term
14	Brightwell Logistics (commission)	ASC 340-40: amortization period extended beyond initial term (non-commensurate renewal rate)
15	Fenwick & Vale Co-Op	Adversarial extraction stress test: contradictory terms, illegible text, missing fields — correct behavior is escalation, not a guessed answer

Definition of “passing” for v1: the system reaches the correct judgment conclusion with correct reasoning and correctly-cited evidence for contracts 1–11 and 13–14; and for contracts 12 and 15, the system correctly identifies and escalates the ambiguity/conflict rather than producing a confident but unsupported answer.

Additional acceptance criteria — billing schedules and contract balances. The test suite exercises contract asset/liability logic without requiring additional fixtures, because the contracts were constructed with divergent billing and recognition patterns:

Contract	Billing/recognition divergence under test
3 — Corvus Manufacturing	Milestone billing (\$100K/\$130K/\$80K) deliberately diverges from cost-to-cost recognition on a combined PO — large swings between contract asset and contract liability; also tests that milestone invoicing does not trigger recognition
4 — Halden Financial	Deferred-start Premium Support billed from Year 2 — Platform and Support obligations produce different balance profiles within one contract, and must net at contract level
7 — Nordholm Industries	Usage fees billed monthly in arrears — revenue is recognized before invoicing, producing a contract asset (or unbilled receivable), the inverse of the typical SaaS deferred revenue pattern
8 — Castellan Public Schools	Prorated fiscal-year-aligned billing with a partial first period — tests proration handling in the billing schedule
11 — Halden Mod C	Cumulative catch-up adjustment must appear as a separately identifiable JE line, not netted into routine recognition

The system passes if it computes the correct net contract asset or contract liability per contract per period, correctly nets at contract level (never presenting both simultaneously for one contract), and never derives the recognition schedule from the billing schedule or vice versa.

9. Open Items for Future Phases (Not v1)

- **Billing schedule push (intended v2 direction).** Rather than Concludo generating invoices, Concludo becomes the **source of truth for the billing schedule** and writes it into the customer's billing system (Stripe, Chargebee, NetSuite, Intacct) as the subscription or billing plan. Because Concludo derives the schedule from the contract and holds the resulting external ID, invoice-to-contract matching is solved by construction rather than by inference — the matching tiers in Section 5.9 become unnecessary. This captures most of the value of owning billing without a tax engine, invoice delivery, or an AR subledger. The cost is that it requires **write** access to the customer's billing system, which is a harder security review and more work per integration than read access, so it should follow rather than precede a proven v1. **Architectural implication for v1: do not model `BillingSchedule` as a read-only artifact of extraction.** Give it a stable identity and a nullable `external_system_ref` from the start, so it can later become an authored, pushable object without a migration.
- Cross-customer benchmark aggregation model: opt-in, anonymized, aggregate-only, with minimum-n thresholds per segment; customers who opt in gain access to enriched peer benchmarks (tiered value exchange). Requires privacy/legal review before build; revisit once a meaningful customer base exists (~15-30+ customers).
- Equity method accounting (ASC 323) as a second judgment domain, reusing the same Contract/Policy/Judgment/Memo architecture.
- Multi-ERP integration framework, generalized after the first real integration is built and proven.
- Consistency gap detection and configurable multi-threshold approval routing.